# Každá 10. s prioritou obsluhy

## Zadání

> Vymyslete příklad systému hromadné obsluhy typu M/M/1, který bude demonstrovat použití priority obsluhy každé 10. transakce. Sestavte jeho model v Petriho síti a SIMLIBu (bez main() a generátoru).

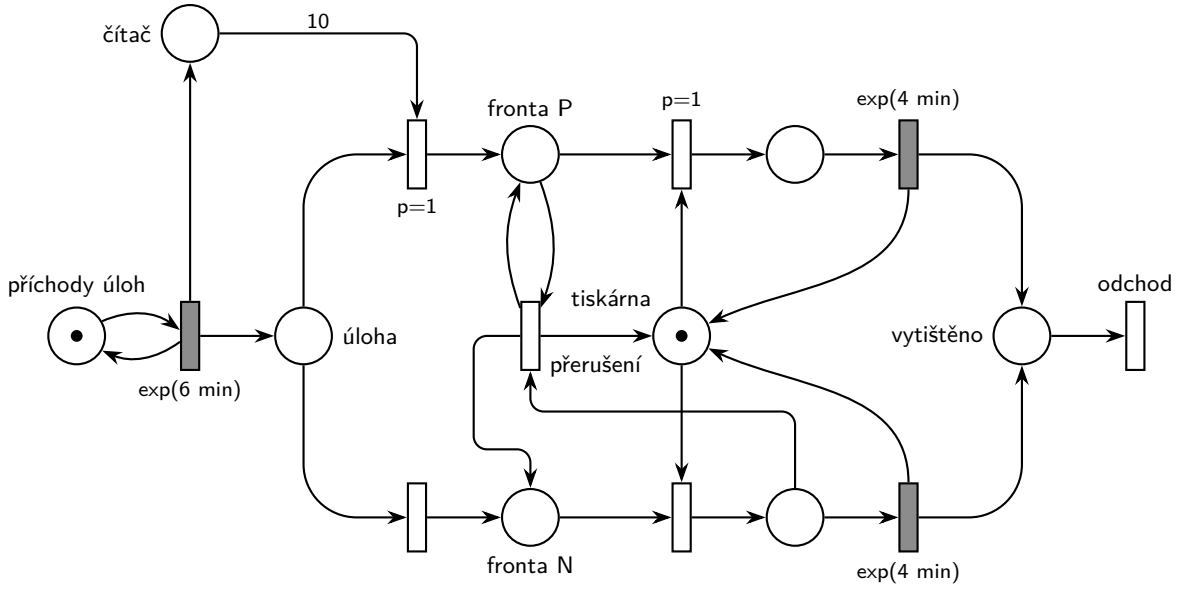
Podobné úlohy:

- Každá N-tá s prioritou procesu: stejný čítač, ale bez přerušení
- Dva proudy příchodů: stejné přerušení, prioritní mají vlastní proud příchodů
- Každá 10. transakce vyvolá poruchu: čítač s hranou váhy 10

## Nápověda

- **Vymyšlený příklad:** tiskárna. Úlohy přicházejí v intervalech `exp(6 min)` (místo *příchody úloh* ve smyčce), tisk trvá `exp(4 min)`. Každá 10. úloha je od vedení a má vyšší prioritu obsluhy (v SIMLIBu `Seize(Tiskarna, 1)`). Když přijde a tiskárna tiskne běžnou úlohu, tisk se přeruší a vytiskne se úloha vedení. Přerušená úloha se pak dotiskne.
- **Čítač** jako v úloze Každá N-tá s prioritou procesu: každý příchod přidá značku do místa *čítač*. Když jich je 10, přechod s hranou váhy 10 (a p=1) pošle právě příchozí úlohu do *fronty P* a čítač vynuluje. Jinak jde úloha do *fronty N*.
- **Tiskárna** je zařízení s kapacitou 1: místo *tiskárna* s jednou značkou mezi oběma řadami. Obsazení z *fronty P* má p=1, oba časované přechody `exp(4 min)` značku vracejí.
- **Priorita obsluhy = přerušení** (slajd 187), ne jen předbíhání ve frontě. Desátá úloha proto potřebuje vlastní frontu a vlastní místo tisku. Jen tak může síť poznat, kterou značku upřednostnit a kterou přerušit.
- **Přerušení ve dvou krocích** jako u poruchy v úloze Porucha s opravou, návrat do fronty. Přechod *přerušení* vezme běžnou úlohu z tisku, vrátí ji do *fronty N* a značku vrátí *tiskárně*. Úloha vedení zůstává ve *frontě P* (hrana tam a zpět). Hned potom ji obsazení s p=1 spojí s volnou tiskárnou. Bez p=1 by si tiskárnu mohla vzít čekající běžná úloha.
- **Přerušená úloha** v SIMLIBu čeká ve vnitřní frontě zařízení a dokončí se přednostně. V síti se vrací do *fronty N*. Pro značení je to totéž, protože síť nerozlišuje běžné úlohy a zbytek exponenciální obsluhy má stejné rozložení jako celá obsluha (podrobněji v úloze Dva proudy příchodů).
- **Jedno křížení:** hrana z tisku N do *přerušení* kříží hranu *tiskárna* → obsazení N. Bez křížení to nejde: s jednou tiskárnou, společným odchodem a přerušením není síť rovinná (na rozdíl od úlohy Dva proudy příchodů, kde mezi dvěma generátory zbývá místo na objížďku).
- Vytištěná úloha **opouští systém** přechodem *odchod*.

## Petriho síť




## Časté chyby

- Jen p=1 na obsazení bez přerušení. To je priorita procesu (úloha ze skupiny G).
- Společná fronta a společné místo obsluhy pro desátou i ostatní transakce (cranycrane): síť pak nepozná, kterou značku přerušit.
- Čítač, který se nevynuluje.
- Přerušená transakce, která zmizí, nebo přerušení, které nevrátí značku *tiskárně*.
- Přerušení i tisku jiné prioritní úlohy: prioritní by se navzájem vyhazovaly.
- V SIMLIBu `Priority = 1` místo `Seize(Tiskarna, 1)` (to je priorita procesu), nebo čítač bez vynulování (cranycrane, 7.cpp).

## SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu. Zadání chce kód bez `main()` a generátoru; tady jsou, aby šel model spustit. Přerušení i návrat přerušené úlohy zařídí zařízení samo díky `Seize(Tiskarna, 1)`; počítadlo `preruseni` jen ukazuje, kdy se v síti provede přechod *přerušení*.

```
// Tiskárna, každá 10. úloha s prioritou obsluhy – model v SIMLIB/C++
#include "simlib.h"

int citac = 0;                                  // místo „čítač“ (počet značek)
long preruseni = 0;                             // kolikrát se provedl přechod
 ↪ „přerušení“

Facility Tiskarna("Tiskárna");                  // místo „tiskárna“ s jednou značkou
Histogram DobaP("Doba úloh vedení v systému [min]", 0, 1, 15);
Histogram DobaN("Doba běžných úloh v systému [min]", 0, 2, 15);

class Uloha : public Process {                  // transakce = značka, která prochází
 ↪ sítí
 public:
  bool desata = false;                          // ve které řadě sítě značka je
  void Behavior() {
    double prichod = Time;
    if (++citac == 10) {                        // příchod přidá značku do čítače;
     ↪ hrana váhy 10 (p=1)
      citac = 0;                                // vynuluje čítač
```

```
      desata = true;
      if (Tiskarna.Busy() && !static_cast<Uloha *>(Tiskarna.In())->desata)
        preruseni++;                          // tiskne se běžná úloha: přechod
 ↪  „přerušení“
      Seize(Tiskarna, 1);                     // priorita obsluhy: přeruší běžnou
 ↪  úlohu,
                                            // ta čeká ve vnitřní frontě přerušených
                                               ↪  (Q2)
    } else
      Seize(Tiskarna);                        // „fronta N“ + obsazení (p=0)
    Wait(Exponential(4));                     // časovaný přechod exp(4 min); u
 ↪  přerušené jen zbytek
    Release(Tiskarna);                        // vrací značku tiskárně
    (desata ? DobaP : DobaN)(Time - prichod);
  }                                           // konec Behavior = přechod „odchod“
};

// zadání main() ani generátor nechce, bez nich ale model nejde spustit
class Prichody : public Event {               // místo „příchody úloh“ ve smyčce s
 ↪  exp(6 min)
  void Behavior() {
    (new Uloha)->Activate();
    Activate(Time + Exponential(6));
  }
};

int main() {
  Init(0, 60 * 10000);                    // modelový čas v minutách: 10 000 hodin
  (new Prichody)->Activate();
  Run();
  Print("Přerušení: %ld z %ld úloh vedení\n", preruseni, DobaP.stat.Number());
  Tiskarna.Output();
  DobaP.Output();
  DobaN.Output();
}
```

| Petriho síť | SIMLIB |
|---|---|
| *příchody úloh* ⇄ `exp(6 min)` | událost `Prichody`, která se znovu plánuje za `Exponential(6)` |
| značka (úloha) | proces `Uloha` |
| místo *čítač*, přechod s hranou váhy 10 a p=1 | `if (++citac == 10) { citac = 0; ... }` |
| místo *tiskárna* (1 značka) | `Facility Tiskarna` |
| *fronta P*, obsazení s p=1 | `Seize(Tiskarna, 1)`: ve frontě Q1 se řadí před běžné |
| *fronta N*, obsazení s p=0 | `Seize(Tiskarna)` |
| přechod *přerušení*: tisk N → *fronta N* + *tiskárna*, test *fronty P* | `Seize(Tiskarna, 1)` u tiskárny obsazené běžnou úlohou: ta jde do fronty přerušených Q2 |
| zaniklý odpočet `exp(4 min)` a nový po návratu na tiskárnu | `Release` vezme přerušenou z Q2 a naplánuje jí zbytek tisku |
| časovaný přechod `exp(4 min)` se sloučeným uvolněním | `Wait(Exponential(4));` `Release(Tiskarna);` |
| přechod *odchod* | konec metody `Behavior()` |

**Kontrola simulací.** Petriho síť: *tiskárna* + tištěné úlohy = 1, *čítač* nepřesáhne 10, všechny přechody se provádějí (`make check`). Prioritní je přesně každá desátá úloha a asi 70 % z nich přeruší probíhající tisk. SIMLIB model: v jednom běhu (10 000 h) byla tiskárna vytížena na 67 % (teoreticky 4/6), přerušení se provedlo u 68 % úloh vedení. Úloha vedení byla v systému v průměru 4,0 min, tedy skoro jen dobu svého tisku; běžná úloha 13,1 min (`make simlib`).

## Původ a zdroje

**Původ:** zkouška 2022/23, 3. termín, skupina H, otázka 2 (body v přepisu nejsou, stejná otázka ve skupině G má 10 bodů). Ve skupině G byla sesterská úloha s prioritou procesu (Každá N-tá s prioritou procesu).

**Zdroje:**

- `Fitcheats/Zkoušky z minulých let/2022-23 - 3. termín H.pdf`, str. 1 (přepsané zadání bez čísel otázek a bodů)
- `Fitcheats/Příprava na zkoušku/cranycrane - Kódy (generátory, Monte Carlo, Euler, RK, Petri + SIMLIB)/petri/7.png` a `7.cpp` (studentské řešení)

**Existující řešení:**

- **cranycrane, 7.png:** `exp(A)` plní frontu i *Counter*. Přechod p=1 bere 10 značek z *Counter*, jednu z fronty a jednu z obsluhy a vrací je do fronty a do obsluhy. Chyby:
  - Desátá transakce nemá vlastní místo. Přechod vrací značky do stejných míst, odkud je vzal, takže se značení fronty ani obsluhy nezmění. Jen se znovu spustí odpočet `exp(B)`, což u exponenciálního rozložení nic nemění. Síť se chová jako obyčejné M/M/1 a žádnou prioritu nemá.
  - Přechod je proveditelný jen při obsazené lince a neprázdné frontě. Přijde-li 10. transakce k volné lince, čítač zůstane plný a přechod se provede později s úplně jinými transakcemi (např. 11. transakce „vyhodí“ z obsluhy tu desátou).
- **cranycrane, 7.cpp:** `if (counter == 10) Seize(linka, 1)`, ale čítač se nikdy nenuluje. Prioritu tak dostane jen úplně první desátá transakce.

## Reference

- Jak síť funguje
- Notace a vzory
- Seznam úloh